Multimodal Structured Generation: CVPR's 2nd MMFM Challenge Technical Report

Franz Louis Cesista
franzlouiscesista@gmail.com

arXiv:2406.11403v2 [cs.CV] 4 Feb 2025

Abstract

Multimodal Foundation Models (MMFMs) have demonstrated strong performance in both computer vision and natural language processing tasks. However, their performance diminishes in tasks that require a high degree of integration between these modalities, such as document understanding. Moreover, fine-tuning these models and deploying them requires significantly more compute and more engineering effort than unimodal models. In this work, we present **Multimodal Structured Generation**, a framework that forces (frozen) MMFMs to produce outputs in a strictly structured format by applying hard constraints directly to the output logits. This approach not only ensures that the model generates parseable outputs that downstream APIs can easily ingest but also allows us to force the model to *reason* before answering, which significantly boosts performance without the need for expensive fine-tuning. We demonstrate the effectiveness of our method through competitive results in the CVPR 2nd MMFM Challenge, highlighting that carefully designed lightweight engineering can outperform expensive and complicated modeling approaches. [1]

1 Introduction

Multimodal Foundation Models (MMFMs) are typically built by grafting together components from different foundation models, each pretrained on a specific modality such as images or text [1]. The resulting "Frankenstein" models have shown impressive results on multimodal tasks where visual and textual data can be processed independently. For instance, in image captioning, a visual encoder first converts an image into a latent representation, which is then passed to a generative language model to produce a caption; similarly, in text-to-image generation, a language encoder transforms text input into a latent representation that an image generation model uses to create an image.

In contrast, document understanding requires the simultaneous processing and integration of visual, textual, and even layout information. And despite the progress made with MMFMs, their performance in tasks demanding deep multimodal integration remains suboptimal. Moreover, adapting these models for more specialized tasks via finetuning is both costly and labor-intensive.

To address these challenges, we introduce **Multimodal Structured Generation**, a framework that forces a (frozen) MMFM to generate outputs that strictly adhere to a predefined structure by applying hard constraints to its output logits. This mechanism allows us to:

1. Force the model to generate outputs that follow the exact format downstream systems or APIs expect, thereby eliminating the need for bespoke post-processing steps. And,
2. Force the model to *reason* before producing an answer. This reduces the need for expensive fine-tuning, thereby reducing computational and engineering costs.

As evidence of the practicality of our approach, we describe how we used this approach to win a prize in the Computer Vision and Pattern Recognition Conference's 2nd Multimodal Foundation Models Challenge. Our team learned about the challenge only two days before the submissions deadline, and we spent the first 24 hours working with a commercially available model that, after clarification with the organizers, we were not permitted to use. With limited time, compute resources, and budget, we could not afford to implement complicated modeling steps. Thus, we turned to our prior work on *Retrieval Augmented Structured Generation* [2].

Retrieval Augmented Structured Generation has four components: (1) Structured Generation [3], (2) Retrieval Augmented Generation [4], (3) Supervised Finetuning, and (4) Structured Prompting [5]. However, for this challenge, we only implemented the Structured Generation component, extending it to multimodal models instead of solely on a language model as in our previous work.

Our approach proved effective: we placed 2nd in Phase 2 of CVPR's 2nd MMFM Challenge and 3rd overall, outperforming several teams that had finetuned their own multimodal models. Notably, Phase 2 featured various never-before-seen document understanding evaluation datasets, demonstrating the generality of our method to unseen tasks. Furthermore, the fact that our method is finetuning-free makes it both cost-effective and easy for other teams to replicate.

[1] All of our scripts, deployment steps, and evaluation results can be accessed in https://github.com/leloykun/MMFM-Challenge

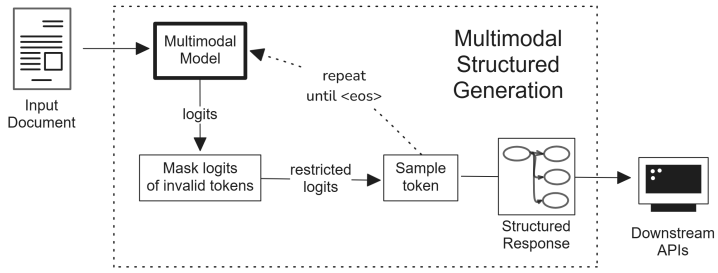

Figure 1: Multimodal Structured Generation

2 Methodology

2.1 Multimodal Structured Generation

Outputs from generative models are not guaranteed to be parseable by downstream programs, compilers, or APIs. For example, if a user requests that a language model generate a Python script for sorting a list of numbers, the output might *seem* runnable yet produce errors when executed in an IDE. To ensure that generated outputs are immediately usable, our approach enforces hard constraints that zero out the logits corresponding to invalid tokens [3]. This method, known as Structured Generation, is readily applicable to multimodal models, which also produce logits that can be constrained to adhere strictly to a predefined schema (see Figure 1).

2.2 Implementation Details

We deploy our multimodal models using Huggingface's Inference Endpoints API and perform inference via Huggingface's Text Generation Interface (TGI) API [6] [7].

For Phase 1, since the test dataset was publicly available and did not fully reflect the generality of our approach, we used an unmodified Llava v1.5 [8]. For Phase 2, which featured never-before-seen evaluation datasets, we augmented our framework with two models. For the `mychart` and `myinfographic` datasets, we employed Llava-Next (v1.6) enhanced with Structured Generation [9] [10]. And to maximize performance on the `mydoc` dataset, we used a finetuned version of Nous Hermes 2 Pro - Mistral 7B[2] augmented with Structured Generation [11].

Our Structured Generation method forces the multimodal models to "reason" before producing an answer. We designed slightly different JSON output formats for each evaluation dataset (available in our repository). For the `myinfographic` and `mychart` datasets, a template can be found in Appendix A.1. For the `mydoc` dataset, we used the entity being requested as the key, following the JSON format from our previous work [2]. This design makes it straightforward to request multiple entities from a document (e.g., `Billing Name` and `Total Amount`). A template can be found in Appendix A.2.

Furthermore, we instructed the models to provide exact answers for the `mydoc` dataset while requesting concise answers for the `mychart` and `myinfographic` datasets. We suspected that the challenge organizers used the MMMU evaluation metric for `mydoc` and judged the outputs for `mychart` and `myinfographic` with the Mistral 7B model as LLM-as-judge [12] [13]. The former requires exact outputs, whereas the evaluation script for the latter indicates that concise responses are preferred. We later learned after the challenge that the Mistral 7B model was used as the judge across all datasets. However, we believe that the exact answers for the `mydoc` dataset contributed positively to our final results.

2.3 Datasets

The MMFM challenge provided the following datasets for Phase 1: IconQA [14], FUNSD [15], WildReceipt [16], TextbookQA [17], TabFact [18], DocVQA [19], InfographicVQA [20], WebSRC [21], and WTQ [22].

3 Results

We evaluated our approach on the two phases of CVPR's 2nd MMFM's Challenge. See Table 1 for results. Our approach placed 2nd in the hidden test set of Phase 2 and 3rd place overall.

[2] This model can be accessed in https://huggingface.co/leloy/Nous-Hermes-2-Pro-Docile-RASG-1ShotRetrieval-StructuredPrompt

Table 1: Evaluation results on CVPR's 2nd MMFM Challenge

	Eval Dataset	Accuracy
Phase 1	iconqa_fill	15.5%
	funsd	32.5%
	iconqa_choose	31.0%
	wildreceipt	35.5%
	textbookqa	51.5%
	tabfact	48.5%
	docvqa	20.5%
	infographicvqa	23%
	websrc	28.5%
	wtq	8.5%
	Phase 1 Overall	**29.5%**
Phase 2	mydoc	62.25%
	mychart	4.5%
	myinfographic	60.98%
	Phase 2 Overall	**50.49%**

Table 2: Ablation Benchmarks of RASG components on KIE & LIR tasks on the DocILE dataset

Model	Key-Information Extraction F1 Score	Line Items Recognition GLIRM-F1
GPT-3.5	34.17%	28.31%
+ 1-Shot Retrieval	+ 22.08%	+ 20.67%
+ Supervised Finetuning	+ 22.31%	+ 17.73%
+ Structured Prompting	+ 4.96%	+ 19.42%
Hermes 2 Pro - Mistral 7B	13.55%	4.69%
+ 1-Shot Retrieval	+ 36.87%	+ 40.55%
+ Supervised Finetuning	+ 17.71%	+ 13.53%
+ Structured Prompting	+ 0.63%	+ 10.30%

* Benchmarks results ablating three components of Retrieval Augmented Structured Generation on Key-Information Extraction (KIE) & Line Items Recognition (LIR) tasks on the DocILE dataset [23]: (1) Retrieval Augmented Generation, (2) Supervised Finetuning, and (3) Structured Prompting. Structured Generation was not included in the ablation benchmarks as it is a necessary component of RASG to ensure that the outputs are parse-able by downstream APIs. Results show that adding Structured Prompting, i.e., adding layout information to the text prompt, only marginally improves performance.

4 Discussion

Our results on the MYDOC dataset are particularly noteworthy: we outperformed multiple teams that finetuned multimodal (vision + text) models by using only an LLM augmented with Structured Generation. This finding reinforces a key insight from our previous work [2]—namely, that for key-information extraction, visual information may be less critical than expected, and in some cases, incorporating vision encoders can even be detrimental.

We propose four hypotheses to explain this observation:

1. The visual and layout information may not be essential for Key-Information Extraction.

   In support of this observation, DocLLM demonstrated that a Text + Layout model can perform as well as, or even outperform, models that integrate both visual and layout cues [24]. Furthermore, our previous work showed that, with appropriate augmentations, entirely omitting layout information does not significantly degrade performance [2]. This finding suggests that the critical factor for successful KIE is the preservation of *locality*—ensuring that words appearing together in the document remain proximate in the text prompt—a property that is naturally maintained by accurate OCR outputs.

2. The LLMs themselves may be capable of inferring the spatial arrangement of words from their sequential positions in the text prompt.

# Tokens Per Grid	Approach	TextVQA	AI2D	ChartQA	DocVQA	MMBench	POPE	ScienceQA	MMMU
576	*SS*	64.53	64.83	59.28	75.40	66.58	87.02	72.29	34.3
	M^3	63.13	66.71	58.96	72.61	67.96	87.20	72.46	34.0
144	*SS*	62.16	65.77	55.28	67.69	67.78	87.66	72.15	36.4
	M^3	62.61	68.07	57.04	66.48	69.50	87.67	72.32	36.1
36	*SS*	58.15	65.90	45.40	56.89	67.01	86.75	71.87	36.2
	M^3	58.71	67.36	50.24	55.94	68.56	87.29	72.11	36.8
9	*SS*	50.95	65.06	37.76	44.21	65.29	85.62	72.37	36.8
	M^3	51.97	66.77	42.00	43.52	67.35	86.17	71.85	35.2
1	*SS*	38.39	63.76	28.96	33.11	61.43	82.83	72.32	35.3
	M^3	38.92	64.57	31.04	31.63	62.97	83.38	71.19	34.8
Oracle	# Tokens	31.39	11.54	41.78	64.09	8.90	6.08	7.43	22.85
	Performance	70.51	76.36	70.76	81.73	74.35	94.29	76.07	50.44

Figure 2: Comparison of approaches with the SS baseline and Matryoshka Multimodal Models (M^3) across various benchmarks under LLaVA-NeXT [27]. The number of tokens (# Tokens) denotes the number of visual tokens per image grid in LLaVA-NeXT. SS denotes the baseline model trained with a Specific Scale of visual tokens. M^3 is at least as good as SS, while performing better on tasks such as TextVQA, ChartQA, and MMBench. Oracle denotes the case where the best tradeoff between visual tokens and performance is picked.

Prior work has shown that LLMs trained without explicit positional encodings can still learn and exploit positional information [25]. It is plausible that, when the text is provided in the order extracted by OCR, the LLM implicitly reconstructs the spatial relationships among words.

3. The limited information capacity of the base language models may also play a role.

   For example, the LLaVA-NeXT (v1.6) model we used is based on the Mistral-7B LLM [9]. A rough estimation using neural scaling laws [26] suggests that such models can store roughly $7 \times 20 = 140$ billion tokens of information. When additional embeddings from the visual encoder and projector are incorporated, the model may quickly reach its capacity limits, thereby reducing its effectiveness for complex document understanding tasks.

4. Perhaps most compellingly, we may not be using enough image tokens for effective document understanding.

   Unlike images of pets, which can be adequately described with just a few tokens, document images are dense with information. Empirically, business documents often contain around 3,000 text tokens—and this number increases when layout details are considered. However, standard image encoders typically compress an entire document into no more than $32^2 = 1024$ tokens, leading to significant information loss. Recent work [27] supports this view, showing that increasing the number of image tokens improves performance on document understanding benchmarks (see Figure 2).

5 Conclusion

Our final standings in CVPR's 2nd MMFM Challenge demonstrate that our approach generalizes effectively to unseen tasks. Notably, our finetuning-free implementation not only reduces computational and engineering overhead, but also makes the method readily reproducible for other teams. While our framework can be extended to finetuned models, our previous work indicates that naively combining finetuning with Structured Generation may actually degrade performance [2]. Future research should further explore the optimal integration of these components, with particular attention to maintaining alignment between output formats in both the finetuning dataset and inference requests.

Additionally, our results on the `mydoc` dataset reinforce the claim that neither detailed visual cues nor precise layout information is critical for effective key-information extraction. Nevertheless, further investigation is needed to rigorously evaluate the four hypotheses proposed in the discussion. Overall, our findings underscore the promise of lightweight engineering approaches in advancing document understanding tasks while reducing reliance on expensive, complex modeling techniques.

References

[1] F. Bordes, R. Y. Pang, A. Ajay, *et al.*, *An introduction to vision-language modeling*, 2024. arXiv: 2405.17247 [cs.LG].

[2] F. L. Cesista, R. Aguiar, J. Kim, and P. Acilo, *Retrieval Augmented Structured Generation: Business Document Information Extraction as Tool Use*, 2024. arXiv: 2405.20245 [cs.CL].

[3] B. T. Willard and R. Louf, *Efficient guided generation for large language models*, 2023. arXiv: 2307.09702 [cs.CL].

[4] O. Ram, Y. Levine, I. Dalmedigos, *et al.*, "In-context retrieval-augmented language models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023, ISSN: 2307-387X. DOI: 10.1162/tacl_a_00605.

[5] W. Wang, Y. Li, Y. Ou, and Y. Zhang, *Layout and task aware instruction prompt for zero-shot document image question answering*, 2023. arXiv: 2306.00526 [cs.CL].

[6] Huggingface Team, *Hugginface Inference Endpoints documentation*, https://huggingface.co/docs/inference-endpoints/index [Accessed: 2024-06-09], 2024.

[7] Huggingface Team, *Hugginface Text Generation Interface documentation*, https://huggingface.co/docs/text-generation-inference/index [Accessed: 2024-06-09], 2024.

[8] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, *et al.*, Eds., vol. 36, Curran Associates, Inc., 2023, pp. 34 892–34 916. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/6dcf277ea32ce3288914faf369fe6de0-Paper-Conference.pdf.

[9] H. Liu, C. Li, Y. Li, *et al.*, *LLaVA-NeXT: Improved reasoning, OCR, and world knowledge*, Jan. 2024. [Online]. Available: https://llava-vl.github.io/blog/2024-01-30-llava-next/.

[10] H. Liu, C. Li, Y. Li, and Y. J. Lee, *Improved baselines with visual instruction tuning*, 2023. arXiv: 2310.03744.

[11] interstellarninja, Teknium, theemozilla, karan4d, and huemin_art, *Hermes 2 Pro - Mistral 7B*, https://huggingface.co/NousResearch/Hermes-2-Pro-Mistral-7B [Accessed: 2024-06-09], 2024.

[12] X. Yue, Y. Ni, K. Zhang, *et al.*, "MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI," in *Proceedings of CVPR*, 2024.

[13] Mistral Team, *Mistral 7B*, https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.1 [Accessed: 2024-06-09], 2024.

[14] P. Lu, L. Qiu, J. Chen, *et al.*, "IconQA: A new benchmark for abstract diagram understanding and visual language reasoning," in *The 35th Conference on Neural Information Processing Systems (NeurIPS 2021) Track on Datasets and Benchmarks*, 2021.

[15] G. Jaume, H. Kemal Ekenel, and J.-P. Thiran, "FUNSD: A dataset for form understanding in noisy scanned documents," in *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*, IEEE, Sep. 2019. DOI: 10.1109/icdarw.2019.10029.

[16] H. Sun, Z. Kuang, X. Yue, C. Lin, and W. Zhang, *Spatial dual-modality graph reasoning for key information extraction*, 2021. arXiv: 2103.14470 [cs.CV].

[17] D. Kim, S. Kim, and N. Kwak, "Textbook question answering with multi-modal context graph understanding and self-supervised open-set comprehension," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 3568–3584. DOI: 10.18653/v1/P19-1347. [Online]. Available: https://aclanthology.org/P19-1347.

[18] W. Chen, H. Wang, J. Chen, *et al.*, "TabFact: A large-scale dataset for table-based fact verification," in *International Conference on Learning Representations (ICLR)*, Addis Ababa, Ethiopia, Apr. 2020.

[19] M. Mathew, D. Karatzas, and C. V. Jawahar, "DocVQA: A dataset for VQA on document images," in *2021 IEEE Winter Conference on Applications of Computer Vision (WACV)*, IEEE, Jan. 2021. DOI: 10.1109/wacv48630.2021.00225.

[20] M. Mathew, V. Bagal, R. Tito, *et al.*, "InfographicVQA," in *2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, IEEE, Jan. 2022. DOI: 10.1109/wacv51458.2022.00264. [Online]. Available: http://dx.doi.org/10.1109/WACV51458.2022.00264.

[21] X. Chen, Z. Zhao, L. Chen, *et al.*, "WebSRC: A dataset for web-based structural reading comprehension," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2021. DOI: 10.18653/v1/2021.emnlp-main.343. [Online]. Available: http://dx.doi.org/10.18653/v1/2021.emnlp-main.343.

[22] A. Drozdov, N. Schärli, E. Akyürek, *et al.*, *Compositional semantic parsing with large language models*, 2022. arXiv: 2209.15003 [cs.CL].

[23] Š. Šimsa, M. Šulc, M. Uřičář, *et al.*, *DocILE benchmark for document information localization and extraction*, 2023. arXiv: 2302.05658 [cs.CL].

[24] D. Wang, N. Raman, M. Sibue, *et al.*, *DocLLM: A layout-aware generative language model for multimodal document understanding*, 2023. arXiv: 2401.00908 [cs.CL].

[25] A. Haviv, O. Ram, O. Press, P. Izsak, and O. Levy, "Transformer language models without positional encodings still learn positional information," in *Findings of the Association for Computational Linguistics: EMNLP 2022*, Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 1382–1390. DOI: 10.18653/v1/2022.findings-emnlp.99. [Online]. Available: https://aclanthology.org/2022.findings-emnlp.99.

[26] J. Kaplan, S. McCandlish, T. Henighan, *et al.*, *Scaling laws for neural language models*, 2020. arXiv: 2001.08361 [cs.LG].

[27] M. Cai, J. Yang, J. Gao, and Y. J. Lee, *Matryoshka multimodal models*, 2024. arXiv: 2405.17430.

A JSON Output Formats

A.1 JSON output format for the `myinfographic` and `mychart` datasets

```
{
  "name": "<tool name e.g. infographic_explair_tool>",
  "description": "<tool description e.g. Infographic Explainer Tool>",
  "parameters": {
    "type": "object",
    "properties": {
      "1_reasoning": {"type": "string"},
      "2_answer": {
        "type": "string",
        "description": "Concise answer to the user question."
      },
    },
    "required": ["1_reasoning", "2_answer"],
  },
}
```

A.2 JSON output format for the `mydoc` datasets

```
{
  "name": "doc_extraction_tool",
  "description": "Extract information from a document",
  "parameters": {
    "type": "object",
    "properties": {
      "1_reasoning": {"type": "string"},
      f"2_{key}": {
        "type": "integer" if key == "page" else "string",
        "description": "The answer, exactly as it appears in the
           document.",
        "maxLength": max_length,
      }
    },
    "required": ["1_reasoning", f"2_{key}"],
  },
}
```

Note that we prepend indices to the keys in the JSON format because the version of TGI we used relies on an older version of Outlines (version $< 0.40.0$) that reorders keys alphabetically. Prepending indices is an effective workaround to ensure the desired order; these indices can be removed in later versions of TGI and Outlines.